TECHNICAL ASSESSMENT

High-Throughput Flash Sale Engine

Senior Backend Engineering Architecture & Implementation Challenge

1. Executive Summary & Context

This technical assessment evaluates your ability to design, implement, and optimize a high-throughput, fault-tolerant backend system using **NestJS**. You are tasked with building the core backend engine for a high-demand flash sale platform (e.g., a limited-edition product drop).

The resulting architecture must elegantly handle extreme concurrent write loads, safely prevent race conditions (guaranteeing you never sell more items than available), enforce execution idempotency across network retries, and gracefully isolate downstream database dependencies.

2. Core Technical Stack

Your solution must explicitly utilize and configure the following infrastructure layers:

- Framework — NestJS (TypeScript) utilizing strict dependency injection and decoupled modular architecture.
- Data Layer — Prisma ORM interacting with a PostgreSQL target database.
- Caching & Concurrency — Redis (leveraging atomic operations, TTL mechanics, and distributed locking).
- Message Broker — RabbitMQ (orchestrating asynchronous task offloading and backpressure control).
- Environment — A unified `docker-compose.yml` file encapsulating all services.

3. System Architecture & Request Lifecycle

4. Detailed Implementation Requirements

4.1 NestJS Core Request Lifecycle Middleware

- **NestJS Guards:** Implement an authentication guard protecting checkout mutations. It must evaluate a simulated session identifier and enforce rate limits.
- **NestJS Interceptors (Idempotency Engine):** Create a specialized `IdempotencyInterceptor`. Every mutation request (`POST`) must require a unique `X-Idempotency-Key` header. The mechanism must guarantee that duplicate requests within a 5-minute window receive the identical cached result without re-executing controller logic.
- **NestJS Pipes:** Implement rigid validation using `ValidationPipe` alongside `class-validator`. Sanitize payloads, enforce type limits, and assert precise UUID patterns.
- **NestJS Exception Filters:** Build a custom global `HttpExceptionFilter` paired with a targeted `PrismaExceptionFilter`. Database exceptions must be gracefully mapped to clean error signatures while preserving internally logged context.

4.2 High-Throughput Inventory & Race Conditions

> ***The Concurrency Problem:*** *When 10,000 users attempt to acquire the final 5 items simultaneously, a naive database update will suffer from locks, timing anomalies, and inventory leakage.*

To solve this, inventory values must be tracked primarily inside **Redis**. You are required to construct an atomic unit of logic using **Redis Lua Scripting** or **Redlock**. This routine must verify stock levels and deduct inventory strictly as an indivisible operation, immediately rejecting overflow requests.

4.3 Asynchronous Storage Offloading via RabbitMQ

Direct database storage under heavy stress degrades connection pool longevity. Your architecture must isolate HTTP response times from disk persistence:

- Upon a successful Redis reservation, the application must immediately return a `202 Accepted` status to the caller.
- Concurrently, an event must be broadcasted to a durable **RabbitMQ** queue.
- A decoupled background consumer must pull tasks from the queue sequentially using a strict prefetch limit and safely update PostgreSQL via **Prisma ORM** within an interactive transaction block.

5. Evaluation Criteria

Metric	Senior-Level Expectations
Concurrency Resilience	Zero over-selling or corrupted state under aggressive simulated user load (500+ requests per second).
Fault Tolerance	Graceful handling of message broker disconnects or unexpected infrastructure degradation without throwing unhandled exceptions.
Idiomatic NestJS	Clean abstraction layers, strict adherence to Dependency Injection, custom decorators, and explicit type hygiene.